

Exercício Prático: API da Feira Online

Turma: LionsDev

Tópicos: APIs REST com Express.js, MongoDB com Mongoose, dois models relacionados,

`ObjectId` + `ref`, `.populate()`, `express.Router()`, Query Params com operadores (`$regex`, `$lte`), validação de ID e Status Codes HTTP (`400`, `404`, `409`).

Nível: avançado. Faça esta lista depois de Academia, Cantina ou Petshop. Ela junta tudo do módulo e adiciona três conceitos novos: separar rotas em arquivos, relacionar dois models de verdade e trazer os dados relacionados com `populate`.

1. Contexto

A **Feira Online** reúne várias barracas de produtores da cidade. Cada barraca vende seus próprios produtos, e hoje ninguém sabe direito o que cada barraca oferece nem quanto custa.

Você deverá criar uma API REST conectada ao MongoDB para cadastrar **barracas** e os **produtos** de cada uma. Um produto sempre pertence a uma barraca.

2. Configuração Inicial e Estrutura de Pastas

Desta vez as rotas **não** ficam todas no `server.js`. Cada recurso ganha seu próprio arquivo de rotas usando `express.Router()`:

```
FeiraOnline/  
|-- models/  
|   |-- barraca.js  
|   |-- produto.js  
|-- routes/  
|   |-- barraca.js  
|   |-- produto.js  
|-- db.js  
|-- server.js  
|-- .env  
|-- package.json
```

Requisitos estruturais:

- Crie um arquivo `.env` contendo as variáveis `MONGO_URI` e `PORT` (porta `3000`).
- O `db.js` conecta ao MongoDB e encerra a aplicação (`process.exit(1)`) se a conexão falhar.
- O `server.js` registra as rotas com `app.use("/barracas", barracaRoutes)` e `app.use("/produtos", produtoRoutes)`.

Conceito novo: `express.Router()`. Um Router é um "mini app" só com rotas. Dentro de `routes/produto.js`, a rota `router.get("/busca")` vira `GET /produtos/busca`, porque o `server.js` montou o arquivo inteiro em `/produtos`.

2.1 Modelos de Dados (Schemas)

Modelo: Barraca (`models/barraca.js`)

- `nome`: Tipo `String`, obrigatório.
- `responsavel`: Tipo `String`, obrigatório.
- `categoria`: Tipo `String`, obrigatório (aceita apenas: `Hortifrúti`, `Laticínios`, `Padaria` ou `Artesanato`).
- `aberta`: Tipo `Boolean`, valor padrão `true`.

Modelo: Produto (`models/produto.js`)

- `idBarraca`: Tipo `mongoose.Schema.Types.ObjectId`, com `ref: "Barraca"`, obrigatório.
- `nome`: Tipo `String`, obrigatório.
- `preco`: Tipo `Number`, obrigatório, não pode ser negativo (`min: 0`).
- `estoque`: Tipo `Number`, valor padrão `0`, não pode ser negativo.

Conceito novo: `ObjectId` + `ref`. Nas listas anteriores, o ID de outro documento era guardado como `String`. Aqui usamos o tipo próprio do MongoDB e dizemos a qual model ele aponta. Com isso, o Mongoose consegue trocar o ID pelo documento inteiro usando `.populate("idBarraca")`.

3. Requisitos Obrigatórios e Regras de Negócio

3.1 Barracas (`routes/barraca.js`)

Método	Rota	O que faz
POST	/barracas	Cadastra uma barraca. Retorna 201 .
GET	/barracas	Lista as barracas. Aceita filtro opcional ? categoria=Padaria .
GET	/barracas/:id/produtos	Retorna a barraca e a lista de produtos dela. 404 se a barraca não existir.
PATCH	/barracas/:id	Abre ou fecha a barraca. Corpo: { "aberta": false } .
DELETE	/barracas/:id	Remove a barraca.

Regras:

- No **PATCH** , o campo **aberta** precisa ser **true** ou **false** . Qualquer outro valor responde **400** .
- Não é permitido remover uma barraca que ainda tem produtos.** Nesse caso, responda **409** (Conflict) informando quantos produtos ela tem. Use **Produto.countDocuments({ idBarraca: id })** .

3.2 Produtos (routes/produto.js)

Método	Rota	O que faz
POST	/produtos	Cadastra um produto. Retorna 201 .
GET	/produtos	Lista os produtos já com nome e categoria da barraca (use populate).
GET	/produtos/busca	Busca por ?nome= (parte do nome, sem diferenciar maiúsculas) e/ou ?precoMax= .
PATCH	/produtos/:id	Atualiza preco e/ou estoque .
DELETE	/produtos/:id	Remove o produto.

Exemplo de corpo do **POST /produtos** :

```
{
  "idBarraca": "66f1c2a9e4b0a1b2c3d4e5f6",
  "nome": "Queijo Minas",
  "preco": 28.5,
  "estoque": 10
}
```

Regras:

1. Só é possível cadastrar produto em uma barraca que **existe** (**404** se não existir) e que está **aberta** (**400** se estiver fechada).
2. Na busca, **precoMax** filtra produtos com preço **menor ou igual** ao valor (**\$lte**). Se **precoMax** não for número, responda **400** .
3. No **PATCH** , atualize apenas os campos enviados e use **{ new: true, runValidators: true }** , para que um preço negativo seja recusado.

3.3 Validação de ID (todas as rotas com **:id**)

Se o ID recebido não estiver no formato do MongoDB (ex.: **/produtos/abc**), responda **400** com a mensagem **"ID inválido."** antes de consultar o banco. Use **mongoose.isValidObjectId(id)** .

Por quê? Sem essa checagem, o **findById("abc")** lança um **CastError** e a API responderia **500** , como se o erro fosse do servidor. O erro é do cliente, então o código certo é **400** .

4. Testes Esperados

1. Cadastre duas barracas de categorias diferentes.
2. Filtre as barracas por categoria.
3. Cadastre três produtos, sendo dois na mesma barraca.
4. Liste os produtos e confira se cada um mostra o nome da barraca (e não só o ID).
5. Busque produtos com **?precoMax=20** e depois com **?nome=queijo** .
6. Feche uma barraca e tente cadastrar um produto nela (esperado: **400**).
7. Tente remover uma barraca que tem produtos (esperado: **409**).
8. Tente buscar um produto com o ID **abc** (esperado: **400**).
9. Remova os produtos de uma barraca e depois remova a barraca (esperado: **200**).

5. Dicas para a Implementação

- A ordem das rotas importa dentro do Router: declare **router.get("/busca")** antes de qualquer **router.get("/:id")** , senão o Express entende **busca** como um ID.
- **populate("idBarraca", "nome categoria")** traz só esses dois campos da barraca.
- Para montar o filtro da busca, comece com **const filtro = {}** e só adicione as chaves que vieram na URL.

- Lembre-se de `app.use(express.json())` no `server.js`.
-

LionsDev • Professor Nicolas Cardoso Motta

Exercício Prático de Mongoose e MongoDB - Módulo 08